软件工程课程综合实验报告

《碳中和主题的城市绿色出行积分系统》

青行 GreenGo | 完整课程实验项目

课程名称	软件工程综合实验
小组编号	（请小组填写）
组长及成员（4—5 人）	（请根据真实人员情况填写）
学院 / 班级	（请填写）
指导教师	（请填写）
报告日期	2026 年 10 月（具体提交日期请填写）

提交说明：本报告包含可运行程序的实现说明、可编辑 UML 源码索引、实际执行的自动化测试结果及 HTTP 业务链证据；团队成员的真实协作和个人贡献须按实际补录。

摘 要

围绕城市碳中和主题，本项目开发一套可实际启动的绿色出行积分教学原型。居民可模拟上报出行数据，系统依据固定教学排放因子计算二氧化碳当量节约量，并将积分以交易台账形式发放；居民可兑换积分商城商品，商户可核销订单，管理员可查看运营统计并进行模拟积分清算。

技术实现采用 FastAPI、SQLAlchemy、SQLite（本地）/MySQL（部署示例），前端采用响应式单页界面；引入事务型 Outbox 实现行程积分发放及订单清算的事件解耦。核心积分以整数计量，余额与库存使用数据库条件更新保证事务一致性。

实际验证：16 项 pytest 测试全部通过，另运行 HTTP 端到端业务链，保存真实 API 响应。限制：碳因子未经官方核证，未接入实际出行数据或支付；Docker 环境与自动浏览器截图没有在本制作环境完成测试。

关键词：绿色出行；软件工程；UML；敏捷开发；事务型 Outbox；积分账本；FastAPI

报告目录

一、实验一：需求分析与迭代规划

二、实验二：系统设计与技术选型

三、实验三：系统原型构建与实现

四、实验四：软件测试与部署

五、工程协作、AI 使用与提交核对

附录：实际运行证据与模型目录

实验一：需求分析与迭代规划

1. 项目背景与范围

本系统面向城市绿色出行激励场景，旨在为参与绿色出行的居民提供可理解、可追溯的积分奖励机制，并为商户及平台管理者提供兑换与清算的管理入口。课程原型只接收模拟行程，不接入公共交通卡、GPS、位置服务或真实碳交易平台，也不将模拟数据宣称为核证减排量。

2. 干系人和主要诉求

参与者	业务目标	核心使用场景	风险与约束
居民	出行获得合理积分、使用权益	登录、行程上报、查余额与流水、兑换	不重复计分，不被超扣，无账户串读
商户	公开商品库存、核销兑换	发布商品、处理订单	库存不能为负，履约必须明确
运营管理员	衡量参与量、核对清算	查询平台指标、清算商户积分	清算只允许已履约订单，重复清算无副作用
项目小组	可迭代、可验证和可复现	UML 建模、Git 协作、自动测试	课程周期和资源有限

3. 用户故事地图

活动	发现/进入	执行业务	确认结果	未来增强
居民出行	登录、了解核算规则	提交模拟行程	查询发放结果、积分流水	GPS 核证、反作弊
居民兑换	浏览和筛选商品	原子扣分、扣库存	查看待履约/已核销订单	取消订单/退款
商户运营	登录商户界面	上架商品、查询订单	确认核销	商户报表与库存预警
平台管理	查看统计	重试事件、查看清算单	已履约订单结算	财务对账、人工稽核

MVP（第一期）的目标故事：居民登录 → 上报公交行程 → 计算减排及发分 → 查积分 → 商城兑换 → 商户核销 → 管理员清算。

4. 功能性需求（可追溯 ID ）

ID	需求	验收准则	重要性
FR-01	登录注册及身份鉴别	错误密码 401；居民可注册；JWT 可访问受保护接口	P0
FR-02	角色权限控制	居民/商户/管理员各自仅访问授权资源	P0

ID	需求	验收准则	重要性
FR-03	模拟行程上报	支持公交/地铁/骑行/步行，0.01—100km，最多 2 位小数	P0
FR-04	碳减排与积分核算	10km 公交节省 1.1kg 并发 11 分；积分取整规则固定	P0
FR-05	行程/事件幂等性	重复行程不上双份积分；相同 ID 内容冲突返回 409	P0
FR-06	积分账本	每一笔发放、消费留有类型、关联业务、正负积分、时间	P0
FR-07	积分商城	所有人可浏览商品和剩余库存	P0
FR-08	安全积分兑换	余额/库存不足拒绝；成功时库存和积分同事务扣减	P0
FR-09	商户商品/订单管理	商户可发布商品并核销所属商品订单	P1
FR-10	平台清算	兑换形成待清算单，已核销才可清算，重复操作幂等	P1
FR-11	平台汇总	可展示注册居民、行程、节约量、发分、订单等指标	P1
FR-12	事件重试	Outbox 事件消费失败后保留，允许再次处理且不双记账	P1

5. 非功能需求

- NFR-01 正确性：积分使用整数记录，账户可用余额不得低于 0；账本汇总与余额一致。

- NFR-02 一致性：成功兑换必须同时完成积分借记、商品库存减少、订单与台账记录；任何步骤失败整体回滚。

- NFR-03 安全性：密码 PBKDF2 加盐哈希、JWT 短期授权、按角色及资源所属校验。演示密码与密钥不得用于生产。

- NFR-04 可维护性：模型、路由、业务服务分层；提供 OpenAPI；基础测试与 GitHub Actions。

- NFR-05 可移植性：本地 SQLite 开箱即用；部署示例 MySQL 8 + Docker Compose（未在当前环境实测）。

- NFR-06 可用性：网页适配桌面与移动屏；缺乏无障碍专项审计，不宣称达到 WCAG 标准。

6. 核心业务计算约束

教学模拟汽车排放基准 180g/km，公交 70g/km，地铁 40g/km，骑行与步行 0g/km。saved_g=floor(distance_m*(180-mode_g/km)/1000)；points=floor(saved_g/100)。非核证碳因子，取整与输入规则详见 README。

7. 四轮 Sprint 计划（建议每轮 1 周 ）

Sprint	目标	完成定义 DoD	可工作增量
S1	确定范围及底座	用户故事、用例图、登录、健康检查、ORM 实体	可登录、可看商品列表
S2	行程与积分闭环	核算规则、幂等性、事件与台账测试通过	可上报并取得积分、查记录
S3	商城与商户	兑换原子事务、履约与清算流程	可从出行到兑换再清算
S4	稳定与交付	集成/并发测试、部署文档、报告与演示材料	可复现完整业务链

注意：上表是课程建议计划，不是声称已经开展了四次真实小组 Sprint。真实会议时间、完成者、评审及回顾由组员实际补充。

8. UML 交付

uml/01-用例图.puml。其他设计与行为模型在 docs/uml/ 中；可用 StarUML/PlantUML 渲染导出图片。

实验二：系统设计与技术选型

1. 架构决策

采用单体模块化 REST 后端 + 事务型 Outbox 事件。Web 端同源 HTML/CSS/JS，无需构建；Python FastAPI 负责 JWT 登录、OpenAPI、参数约束，SQLAlchemy ORM 负责事务和数据库结构。本地 SQLite 便于课程复现；Docker 示例切换 MySQL 8。相较于微服务+消息队列，单体 Outbox 的搭建负担小，且能展示“事务内产生事件，独立消费可重试”的 EDA 思想。

实际事件处理：submit_trip 在事务中同时保存行程和 Outbox；提交之后同步调用 process_pending 快速响应；worker.py 提供异步轮询恢复路径。生产级多消费者队列、事件租约及死信管理尚未实现，故应限定为课程级单机演示方案。

2. 模块设计

- app/main.py：路由与输入约束、用户身份、角色权限、序列化输出。
- app/models.py：数据库实体和唯一约束。
- app/services.py：整数碳减排计算、原子兑换、幂等处理、事件消费、聚合统计。
- app/security.py：PBKDF2 密码哈希、JWT 签发和校验。
- app/static/：浏览器单页客户端，包含登录、居民看板、行程、商城、积分流水、订单、商户及管理视图。
- scripts/worker.py：轮询待处理事件，实现失败重试能力。

3. 据模型（ 行源一致 ）

表	主键/核心字段	关键约束/关系
users	id, username, password_hash, role, balance	username 唯一；balance 整数
trips	id, user_id, client_trip_id, mode, distance_m, saved_g, points, status	(user_id,client_trip_id)唯一
point_transactions	id, user_id, kind, reference_id, delta	(kind,reference_id)唯一，业务幂等键
products	id, merchant_id, cost, stock, active	商户关联；库存非负由条件更新保证
redemptions	id, user_id, product_id, points, status	关联商品/居民，保留兑换时商品名和成本快照
settlements	redemption_id, merchant_id, points, status, settled_at	与兑换单 1:0..1 关联；清算单由事件产生
outbox_events	id, kind, ref_id, state, attempts, error	(kind,ref_id)唯一；消费失败可重试

4. 积分一致性设计

1. 行程业务键 (user_id,client_trip_id) 唯一，避免用户反复提交同编号行程。

2. 减排量保存为 saved_g 整数、距离为 distance_m 整数，积分为整数。不存在浮点小数积分。

3. Outbox 与新行程在同事务提交；消费者发放余额变动、台账和行程状态在同一事务。

4. point_transactions 的 (kind,reference_id) 唯一，事件重放不能形成重复积分流水。

5. 兑换时使用条件 UPDATE：users.balance>=cost 与 products.stock>=1；成功时生成订单、负向台账和新 Outbox 事件，共同提交。

6. 清算只允许 FULFILLED 订单，重复清算直接返回已结算而不再计入积分。

5. 逻辑状态机

- 行程：PENDING → AWARDED（成功发放）。事件处理失败则保持 PENDING，等待重试。

- 订单：PROCESSING → TO_FULFILL → FULFILLED。

- 清算：PENDING → SETTLED。清算不等于真实货币转账，仅记录积分结算状态。

- Outbox：PENDING → DONE；失败增加 attempts 并保留待重试状态。

6. 主要接口

HTTP 路径	身份	功能
POST /api/auth/login	公共	颁发 JWT
POST /api/auth/register	公共	注册居民
GET /api/me	登录	查询当前用户
GET /api/rules	公共	查看演示核算系数
POST /api/trips	居民	模拟上报行程并触发发分
GET /api/trips	居民	查询本人行程
GET /api/wallet	居民	查询积分余额和流水
GET /api/products	公共	浏览在售商品
POST /api/redemptions	居民	原子积分兑换
GET /api/redemptions	居民	查看本人订单
POST /api/merchant/products	商户	创建商品
GET /api/merchant/orders	商户	查看本商户订单

HTTP 路径	身份	功能
POST /api/merchant/orders/{id}/fulfill	商户	核销订单
GET /api/admin/stats	管理员	查看平台指标
GET /api/admin/settlements	管理员	查看清算账
POST /api/admin/settlements/{id}/settle	管理员	清算已核销订单
POST /api/admin/events/process	管理员	触发待处理事件重试

接口完整参数及响应实例以启动后的 /docs 自动生成 OpenAPI 为准。

7. UML 模型

uml/02-领域类图.puml、03-行程积分时序图.puml、04-兑换清算时序图.puml、05-订单状态图.puml、06-组件部署图.puml。报告附相应架构/ER 可视化预览，PlantUML 源文件可继续编辑。

8. 技术风险与取舍

- 演示环境：SQLAlchemy create_all 适合第一次建库，生产演进需 Alembic 迁移版本控制。
- 演示账户：默认演示密码和密钥仅供本地课程，不能暴露在公网上。
- 并发：通过事务条件 UPDATE 处理扣减竞争，SQLite 并发集成用例通过；尚无压力测试报告。
- 碳核算：固定系数只演示计算算法，实际需核验路线、时间、出行凭证和地区因子。
- 审计：未实现订单退款、风控、完整操作日志、限流或财务支付。

设计图示（与源码模型对应）

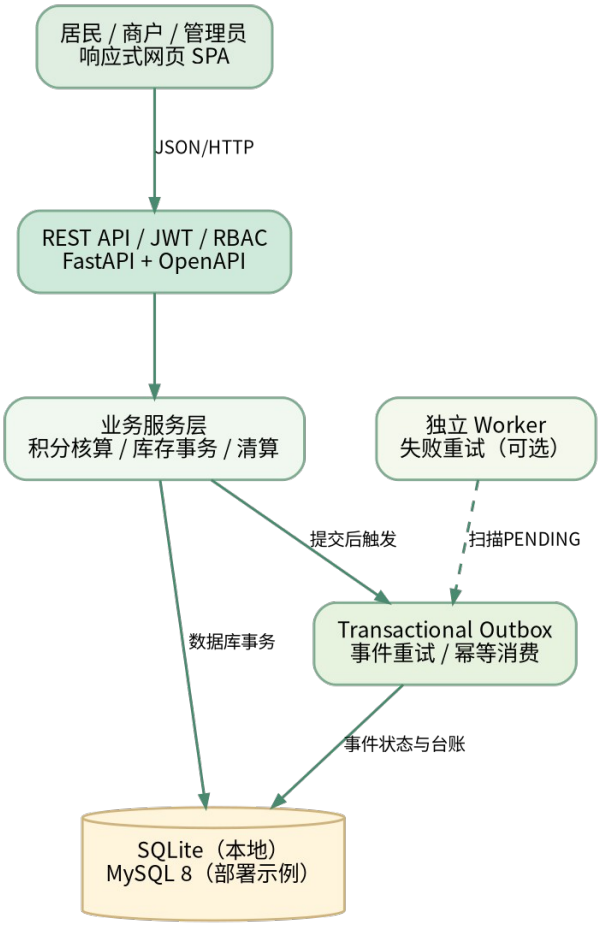


图 1 模块化后端与事件处理部署视图（结构化预览）

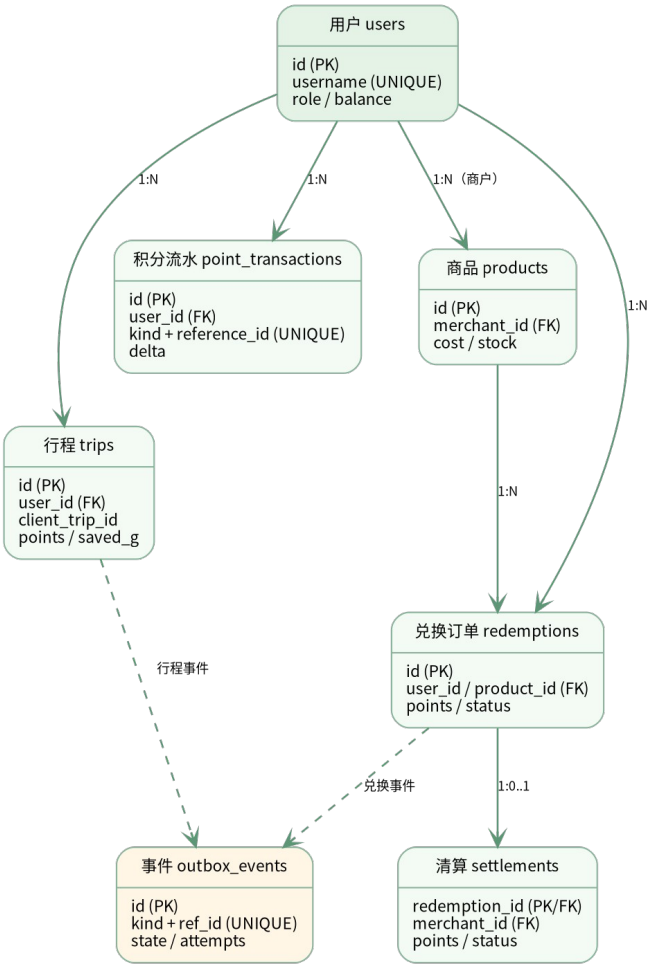


图 2 主要实体及关联关系（结构化预览）

说明：上述预览图由 Graphviz 绘制；符合 UML 表述的可编辑用例图、类图、时序图、状态图、组件图源文件位于 docs/uml/。

实验三：系统原型构建与实现

1. 原型功能

启动后端即可从网页端访问：

1. 登录页面：切换居民、商户、管理员演示账号，认证成功后按角色显示菜单。
2. 居民工作台：账户积分、模拟减排、行程数量及里程、最近 7 天积分柱状趋势。
3. 行程上报：选择方式与距离，实时预估碳量与积分，自动生成防重行程号。
4. 积分商城：6 个初始化商品，按品类筛选、显示库存与积分价格，并确认兑换。
5. 积分账本：查看正负流水与业务关联信息。
6. 兑换记录：显示订单履约状态。
7. 商户工作台：上架商品、查看订单、点击核销。
8. 管理后台：统计信息、清算单、已核销订单清算、Outbox 事件重试。

2. 示例用例及计算

POST /api/trips {"client_trip_id":"BUS-0001","mode":"bus","distance_km":"30"}：

- distance_m=30000
- saved_g=30000 × (180−70) / 1000 = 3300g，即 3.3kg
- points = floor(3300/100)=33
- 提交行程与 TripSubmitted 事件后，事件消费者写入 +33 流水与余额，并将行程标记 AWARDED。

商品价格 12 分，兑换事务执行条件扣减余额及库存，创建兑换单-12 流水与 RedemptionCreated 事件。商户核销后运营可清算该订单。

3. 源文件与可运行环境

README.md 是主运行指导文档；requirements*.txt 列出了运行和测试依赖；Dockerfile 与 docker-compose.yml 包含容器部署样例。实测使用 Python 3.13、FastAPI 0.128、SQLAlchemy 与 SQLite 的本地配置。前端使用原生 JavaScript 的 Fetch API，浏览器同源访问 REST 接口。

4. 关键代码说明

- calculate()：先把最多两位小数公里值转换为整数米，按整数算出减排克数、积分；非法输入直接拒绝。
- submit_trip()：使用用户与行程编号唯一性，保持重复上报的幂等行为。

- process_pending()：处理 TripSubmitted/RedemptionCreated；以流水业务键和事务保障消费重放安全。

- redeem()：UPDATE ... WHERE balance >= cost 与 UPDATE ... WHERE stock >= 1 处于单数据库事务，任一检查失败则整体回滚。

- require_role()与资源所属判断：对商户订单查询、核销、居民流水等提供权限约束。

5. 敏捷迭代与 Git 落地

项目目录可以直接初始化 Git 并按真实修改提交，已配置.gitignore 排除数据库、环境密钥等。仓库的本地示例提交不是 4—5 名成员分别提交的记录。正式课程团队需采用 Feature Branch、Pull Request 合并、每周 Sprint 评审与真实 Commit 日志；README、docs、tests 与代码共同纳入 Git 版本。

建议看板列：Backlog → Todo → In Progress → Review → Done。用户故事对应验收结果和自动测试，Sprint 结束时以可运行的服务而不是仅静态文档作为增量。

6. 演示证据

evidence/demo_flow.json 通过 Python requests 对运行中的真实 Web 服务器实际发起 HTTP 请求而生成，包含业务响应、订单状态与最终账本；evidence/pytest-output.txt 为 pytest 原始控制台输出。自动浏览器截图工具受当前系统强制 URL 屏蔽策略限制，因此没有把不可验证的合成图片称为运行截图。

实验四：软件测试与部署

1. 测试目标

验证业务计算规则、访问控制、幂等性、积分与库存原子扣减、订单履约及清算处理。测试采用 pytest + FastAPI TestClient + SQLAlchemy SQLite。集成测试每次重新建立实际数据库表结构，向 REST API 发送请求，非仅 Mock 返回值。

2. 核心用例设计和已执行结果

测试编号	对应测试函数	输入/场景	预期	结果
TC-01	test_health_docs_and_static	健康检查、首页、Swagger	200 / 健康	PASS
TC-02	test_login_wrong_credentials	错误密码/无 token	401	PASS
TC-03	test_register_role_and_duplicate_username	注册/重复用户名	201 / 409	PASS
TC-04	test_calculation_boundaries	公交 10km、地铁、骑行、0 公里、超限	精确积分或拒绝	PASS
TC-05	test_trip_awards_and_ledger	公交 10km 上报	1100g、11 分	PASS
TC-06	test_trip_idempotency_and_conflict	同编号重复/改数据	不双记账 / 409	PASS
TC-07	test_trip_validation	不合法交通方式/距离	4xx	PASS
TC-08	test_no_free_or_negative_points	步行 0.01km	0 分，不负债	PASS
TC-09	test_redemption_insufficient_balance	零积分兑换	409，不扣库存	PASS
TC-10	test_redemption_success_and_atomic_ledger	+33 积分兑换 12 分	余额 21,负向流水,库存-1	PASS
TC-11	test_sold_out_does_not_charge	库存 0 下兑换	409，积分不变	PASS
TC-12	test_roles_and_data_isolation	跨角色及跨居民读取	403/隔离	PASS
TC-13	test_event_retry_is_idempotent	回置事件 PENDING 再处理	仅一条发放流水	PASS

测试编号	对应测试函数	输入/场景	预期	结果
TC-14	test_full_merchant_fulfillment_settlement	先清算/商户核销/重复清算	409/成功/幂等	PASS
TC-15	test_merchant_can_publish_product	商户新增商品	201 且列表可见	PASS
TC-16	test_atomic_stock_for_concurrent_requests	双线程抢仅存 1 件商品	一成功一 409，未超卖	PASS

实际测试输出： 16 passed（详见 docs/evidence/pytest-output.txt）。此结果证明的是当前隔离环境中覆盖到的功能，不代表系统不存在其他缺陷。

3. HTTP 业务链复验

启动 uvicorn app.main:app --port 8000，运行 python scripts/demo_flow.py（本次取证时通过 DEMO_BASE_URL 指向 8765 测试端口）：

- POST /api/trips 模拟 30km 公交，返回 +33 分；
- POST /api/redemptions 兑换 12 分商品，订单转为待核销；
- 商户核销完成后，管理员清算结果 SETTLED；
- 另存实际调用结果在 docs/evidence/demo_flow.json（依环境中既有示例行程，最终余额可能高于本次净增 21 分）。

4. 部署架构与步骤

本地：安装依赖，配置 JWT_SECRET，运行 uvicorn app.main:app --host 0.0.0.0 --port 8000，数据库首次自动生成；访问浏览器和 /docs。

Docker Compose 示例（未在当前工作环境实际部署）：docker compose up --build -d。配置 MySQL8 容器、FastAPI API 容器与 Outbox worker 容器，建立 MySQL 健康检查与持久卷。对公网发布之前需要替换默认凭证/密码并接入 HTTPS 反向代理、密钥管理与备份。

操作注意事项：SQLite 单机适合课程演示，多容器与并发推荐 MySQL；需明确部署节点与 HTTPS 入口，生产应创建非演示管理员并禁用固定凭证。当前 Docker 引擎不可用，故未声称执行过真实容器部署。

5. 未覆盖及未来增强

1. 缺少真实出行证明：防作弊须接入行程证据与反刷校验，否则用户可以任意输入距离。
2. 缺少退款闭环：订单取消、商品退货、积分反冲与库存归还应独立建模并测试。
3. 事件可靠性扩展：多 Worker 需队列领取锁、重试上限与死信箱；当前课程原型以简单待处理重试演示。

4. 缺少全面质量指标：未进行真实浏览器自动化 UI 截图、跨浏览器兼容测试、正式负载压测和真实 MySQL 容器测试。

5. 安全生产化：需补强限流、CSRF 策略评估、HTTPS、生产用户配置、审计、备份及依赖安全扫描。

6. AI 使用和人工审核说明

本交付中，AI 辅助进行了需求与 UML 初稿、后端及网页原型生成、测试设计与报告组织；已通过运行 pytest 以及执行 HTTP 链路进行部分自动验证。小组最终使用时必须重新逐行理解/审查核心规则与设计、补充业务场景、注明每个人的实际人工修改及测试，并对教师答辩的设计依据负责。这里的“人工审查情况”不能预填为已经由组员审查通过。

五、工程协作、AI 使用与提交核对

1. 真实团队分工与 Git（提交前必）

以下表格是课程互评模板，不代表本交付已经由 4—5 名学生协作完成，也不是基于真实 Git 数据的评分。请根据实际 GitHub/Gitee 提交与 Issue/PR 记录补全。

成员（真实姓名）	负责模块/任务	Git 提交与 PR 证据	组内评分（待评）
组长：待填	待填	待填	待填
成员 2：待填	待填	待填	待填
成员 3：待填	待填	待填	待填
成员 4：待填	待填	待填	待填
成员 5（可选）	待填	待填	待填

提交依据：Git 仓库 URL、真实 commit 哈希及时间、Sprint 看板截图、同伴评语和任务验收记录。不得把 AI 生成的本地示例提交归功于虚构成员。

2. AI 使用情况及后续人工审核计划

开发环节	AI 辅助内容（已发生）	已进行的机器验证	人工检查/修改（小组待填）
需求分析	功能点、用户故事及范围界定初稿	文档结构与代码对照	逐条复核需求
UML 与设计	模型文字草图及 PlantUML 源文件	校对实体与数据库字段	人工审图并修订
代码实现	后端、前端和数据表原型	16 项 pytest 通过、HTTP 链实测	逐行审核核心逻辑、重构
测试	正向/反向/并发测试用例	pytest 结果存档	补充遗漏场景及测试记录

开发环节	AI 辅助内容（已发生）	已进行的机器验证	人工检查/修改（小组待填）
部署与报告	说明文档及容器配置建议	本地 API 验证	部署到小组实际环境并记录截图

3. 最收交付

- [已提供] 前后端可运行源码和静态资源
- [已提供] 数据表与 ORM 结构及演示数据库
- [已提供] 6 份可编辑 PlantUML 模型与图示预览
- [已提供] 敏捷 Sprint 计划、需求分析、设计说明、测试报告
- [已提供] 16 项自动测试与真实 HTTP 请求证据
- [已提供] Dockerfile / Compose 配置（未在本环境实际执行）
- [需组员补全] 真实协作 Git 仓库及分工互评
- [需组员补全] 实际 Sprint 评审与小组答辩记录

附录：实际运行证据与模型目录

A. 自动化测试输出

```
.....                                                    [100%]
16 passed in 3.81s
```

B. HTTP 业务链测试数据

步骤	接口	HTTP 状态
登录 resident	POST /api/auth/login	200
提交 30km 公交	POST /api/trips	200
读取积分余额	GET /api/wallet	200
兑换 12 积分商品	POST /api/redemptions	200
登录 merchant	POST /api/auth/login	200
商户查询订单	GET /api/merchant/orders	200
商户核销	POST /api/merchant/orders/2/fulfill	200
登录 admin	POST /api/auth/login	200
管理员清算	POST /api/admin/settlements/2/settle	200

步骤	接口	HTTP 状态
查询最终积分流水	GET /api/wallet	200
全站统计	GET /api/admin/stats	200

实际本次模拟公交行程获得 33 积分，兑换支出 12 积分；运行时账户余额 42 分，当前账本记录累计差额 42 分。原始请求响应存储于 docs/evidence/demo_flow.json。运行时账户余额包含先前的演示操作，不能误读为单笔出行所得。

C. UML 源文件索引

- docs/uml/01-用例图.puml
- docs/uml/02-领域类图.puml
- docs/uml/03-行程积分时序图.puml
- docs/uml/04-兑换清算时序图.puml
- docs/uml/05-订单状态图.puml
- docs/uml/06-组件部署图.puml

D. 重要真实性说明

测试结论只描述本次本地执行结果，不能推断在线支付、真实碳核证、真实团队协作、Docker 部署或生产级安全符合要求。对于课程最终验收，应安排各组员在自己的设备上运行、复测、审查、补全报告的个人贡献及使用情况。

附录 E 功能扩展：人工客服与大模型绿色出行助手

版本说明：本附录记录 2026 年 10 月 20 日扩展的两项真实软件功能。前文的原实验记录保留为对应迭代时的证据；本附录测试结果是功能扩展后的重新执行结果。

E.1 新增用户故事与功能范围

需求编号	功能描述	实现与约束
CS-01	用户/商户在线工单咨询	分类、主题、内容输入；后台 SQLite 持久化；仅工单创建者及管理员可访问
CS-02	管理员人工客服处理	查看所有工单、追加消息、状态变更；用户继续回复可重新打开已解决工单
AI-01	大模型绿色出行问答	后端调用配置的 OpenAI 兼容 Chat Completions 服务；浏览器不接触 API 密钥
AI-02	未配置模型的处理	明确显示“未配置大模型”，禁用发送；右侧 FAQ 为非 AI 固定说明

E.2 数据库与架构设计

数据库新增 support_tickets 与 support_messages 两张表。前者记录工单所有者、分类、主题、状态、创建和更新日期；后者按工单编号保存每条回复及实际发送者。工单状态：OPEN（待处理）、IN_PROGRESS（处理中）、RESOLVED（已解决）。原行程、积分、商品、清算表与演示数据不做破坏性迁移；首次启动由 SQLAlchemy 在原数据库上自动新增表。

大模型模块采用“浏览器→FastAPI→大模型适配器→HTTPS 外部模型服务”的分层设计。用户问题与最多 8 条历史传入后端，由系统提示词加入模拟碳核算规则，再由 httpx 调用供应商标准接口。服务端环境变量 AI_API_KEY、AI_BASE_URL、AI_MODEL 控制真实模型；大模型不具有写入积分、订单或清算数据的权限。

E.3 实验三：功能实现与验证方法

客服入口：居民/商户菜单“客服中心”→新建咨询→管理员登录查看工单→回复/标记已解决→居民再次登录查看并补充消息。工单及回复跨账户切换保留在数据库。AI 入口：“AI 出行助手”→检查模型配置→有真实 API Key 时可进行多轮问答；未配置时提供明确提示和本地 FAQ，杜绝冒用 AI 回复。

E.4 实验四：测试结果（功能扩展版本）

测试组成	本次结果
原积分商城与事件清算业务测试	16 项通过
原登录界面显示控制测试	3 项通过
新增客服与大模型 API 集成测试	8 项通过
合计自动化测试	27 项通过（pytest，隔离临时 SQLite 数据库）
Chromium 前端 DOM 交互模拟	4 个关键场景通过，页面 JavaScript 错误 0

新增测试覆盖：工单持久化与状态流转、不同用户间的工单隔离、管理员权限、非法输入、大模型未配置状态、兼容接口请求体、供应商错误信息脱敏及不安全 URL 拒绝。大模型测试通过模拟的 HTTP 供应商返回值完成，不等于已经使用真实收费模型成功联网。

E.5 部署配置、隐私与使用 AI 的说明

运行环境：Windows Anaconda Prompt；与前文同一个 greengo Conda 环境。应用仍用 python -m uvicorn app.main:app --reload 启动。必须先由使用者申请合法 API Key，并在 Anaconda Prompt 设置 AI_API_KEY、AI_MODEL 和 AI_BASE_URL，模型权限和额度由供应商决定。示例 DeepSeek 配置：AI_BASE_URL=https://api.deepseek.com；AI_MODEL=deepseek-flash（以供应商当前支持情况为准）。

客服消息持久化于平台数据库；AI 历史仅在网页会话期间保存在浏览器内存，只有用户主动发送后才会将输入文本转发给配置的模型服务；不得录入账号密码、密钥等敏感信息。大模型输出仅作规则解释，不代替准确的服务器积分计算。

研发过程使用 AI 工具辅助生成部分模型、代码和测试初稿，已进行人工结构调整及可复现的自动测试。用户个人真实 API Key 联网验证、团队成员各自的人工审查、联调和答辩仍应在实际提交前补齐并如实记录。

E.6 新增模型及操作文档索引

- docs/uml/07-客服工单时序图.puml
- docs/uml/08-客服工单状态图.puml
- docs/uml/09-大模型适配器组件图.puml
- docs/05-客服中心与大模型集成.md
- 客服与 AI 功能操作指南.md